Métodos Cuantitativos

Clase 14 - Selección de variables en modelos cuantitativos

Unidad III: Regresión Logística

Introducción

- Hasta el momento, hemos aprendido a ajustar modelos tanto para problemas de regresión (cuando la variable dependiente es continua) como para problemas de clasificación (cuando la variable dependiente es discreta).
- Sin embargo, surge una pregunta fundamental en el proceso de modelado:
  - **¿Cómo decidimos qué variables deben permanecer en nuestro modelo y cuáles se pueden descartar sin perder poder explicativo o predictivo?**

Introducción

- Aquí cobra relevancia el **principio de parsimonia**, también conocido como la **navaja de Occam**.
- Este principio establece que, entre dos modelos con un desempeño similar (explicativo o predictivo), se debe preferir aquel que sea más sencillo, es decir, que utilice la menor cantidad de variables posible. El objetivo es evitar la complejidad innecesaria en el modelo.

Introducción

- El objetivo es **evitar la complejidad innecesaria en el modelo**, obteniendo como resultado:
  - Reduce el riesgo de **sobreajuste** (overfitting).
  - Facilita la **interpretación** y la **comunicación de los resultados**.
  - Suele ser más **robusto** ante datos futuros.

Introducción

- En esta sesión, revisaremos cómo hacer una selección de variables basada en criterios estadísticos formales, utilizando:

  1. **Prueba t** para evaluar la significancia individual de los coeficientes.
  2. **Selección de variables según métricas**.
  3. **Criterios de información** (AIC y BIC) para comparar la bondad de ajuste y la complejidad de distintos modelos.

¿Qué veremos hoy?

- **Prueba t para los coeficientes**
- Selección de variables por métrica
- Criterios de información: AIC y BIC

Prueba t para los coeficientes

Prueba t para los coeficientes

- En el contexto de regresión lineal o logística, la **prueba t** se utiliza para evaluar si un coeficiente β_i es significativamente distinto de 0. La hipótesis nula es:
  - $H_0 : \beta_i = 0 \quad$ (la variable no tiene efecto)
  - $H_1 : \beta_i \neq 0 \quad$ (la variable sí influye en la respuesta)

Prueba t para los coeficientes

- Para cada variable X_i, `statsmodels` nos entrega en el resumen (con la función `summary()`) los **coeficientes estimados**, el **t-statistic** y el **p-value** asociado.
  - **Si el p-value** $<$ α (por ej., $\alpha = 0.05$), rechazamos H_0 y concluimos que la variable X_i es significativa en el modelo.
  - **Si el p-value** $>$ α, no hay evidencia estadística suficiente para afirmar que β_i sea diferente de 0.

Prueba t para los coeficientes

- En la práctica, para la selección de variables, la prueba t se emplea principalmente en los métodos **Backward y Forward** (así como en Stepwise):

  1. **Backward Elimination**: se elige la variable con el p-value **más alto** (menos significativa) y se elimina si supera el umbral α.
  2. **Forward Selection**: se agrega la variable que tenga el p-value **más bajo** (o que más mejore un criterio de información), siempre que sea $< \alpha$.
  3. **Stepwise**: combina ambas ideas; en cada paso, se revisan los p-values de las variables que están dentro (para potencialmente eliminarlas) y de las que están fuera (para potencialmente agregarlas).

Prueba t para los coeficientes

Ejemplo con Python

Para el ejemplo ocuparemos el dataset de diabetes:

```python
from sklearn.datasets import load_diabetes
import pandas as pd

X, y = load_diabetes(as_frame=True, scaled=False, return_X_y=True)
data = pd.concat([X, y], axis=1)
print(data)
```

```
      age  sex   bmi      bp     s1     s2    s3    s4      s5     s6  target
0    59.0  2.0  32.1  101.00  157.0   93.2  38.0  4.00  4.8598   87.0   151.0
1    48.0  1.0  21.6   87.00  183.0  103.2  70.0  3.00  3.8918   69.0    75.0
2    72.0  2.0  30.5   93.00  156.0   93.6  41.0  4.00  4.6728   85.0   141.0
3    24.0  1.0  25.3   84.00  198.0  131.4  40.0  5.00  4.8903   89.0   206.0
4    50.0  1.0  23.0  101.00  192.0  125.4  52.0  4.00  4.2905   80.0   135.0
..    ...  ...   ...     ...    ...    ...   ...   ...     ...    ...     ...
437  60.0  2.0  28.2  112.00  185.0  113.8  42.0  4.00  4.9836   93.0   178.0
438  47.0  2.0  24.9   75.00  225.0  166.0  42.0  5.00  4.4427  102.0   104.0
439  60.0  2.0  24.9   99.67  162.0  106.6  43.0  3.77  4.1271   95.0   132.0
440  36.0  1.0  30.0   95.00  201.0  125.2  42.0  4.79  5.1299   85.0   220.0
441  36.0  1.0  19.6   71.00  250.0  133.2  97.0  3.00  4.5951   92.0    57.0

[442 rows x 11 columns]
```

Prueba t para los coeficientes

Ejemplo con Python

Ajustamos un modelo de regresión:

```python
import statsmodels.api as sm
from sklearn.model_selection import train_test_split

# generamos la matriz de atributos
X = data.drop(columns="target")
# generamos el vector respuesta
y = data['target']

# separamos los datos en entrenamiento y validación
X_train, X_test, y_train, y_test = train_test_split(
  X, y, random_state=42, test_size=0.2
)

# añadimos el intercepto a la matriz de entrenamiento
X_train = sm.add_constant(X_train)
# ajustamos el modelo
modelo = sm.OLS(y_train, X_train)
resultados = modelo.fit()
```

Prueba t para los coeficientes

Ejemplo con Python

Reportamos la tabla resumen:

```python
print(resultados.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                 target   R-squared:                       0.528
Model:                            OLS   Adj. R-squared:                  0.514
Method:                 Least Squares   F-statistic:                     38.25
Date:                Sun, 08 Mar 2026   Prob (F-statistic):           5.41e-50
Time:                        14:20:15   Log-Likelihood:                -1906.1
No. Observations:                 353   AIC:                             3834.
Df Residuals:                     342   BIC:                             3877.
Df Model:                          10
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       -341.3782     74.073     -4.609      0.000    -487.074    -195.683
age            0.1377      0.251      0.549      0.583      -0.356       0.631
sex          -23.0645      6.536     -3.529      0.000     -35.921     -10.208
bmi            5.8464      0.829      7.049      0.000       4.215       7.478
bp             1.1971      0.246      4.873      0.000       0.714       1.680
s1            -1.2817      0.621     -2.065      0.040      -2.503      -0.061
```

Prueba t para los coeficientes

Ejemplo con Python

- En la tabla de los β_i, se puede observar el campo **P>|t|**, este nos indica si β_i es **estadísticamente diferente de 0**.
- Esta información también se puede obtener con:

```python
# agregamos round para eliminar las notaciones cientificas
print(round(resultados.pvalues, 4))
```

```
const    0.0000
age      0.5834
sex      0.0005
bmi      0.0000
bp       0.0000
s1       0.0397
s2       0.1557
s3       0.4836
s4       0.1385
s5       0.0002
s6       0.5079
dtype: float64
```

Prueba t para los coeficientes

Ejemplo con Python

```
valores_p = resultados.pvalues
print(valores_p[valores_p.values > 0.05])
```

```
age    0.583439
s2     0.155705
s3     0.483573
s4     0.138466
s6     0.507926
dtype: float64
```

- En este ejemplo, las variables **no significativas** serían age, s2, s3, s4 y s6, por lo que podrían eliminarse del modelo.
- Sin embargo, hay que ser precavidos al **eliminar todas las variables no significativas de forma simultánea**, pues es posible que una variable resulte no significativa únicamente en presencia de otra que sí lo es.
- Por ello, conviene utilizar estrategias stepwise que añadan o eliminen variables de manera iterativa, analizando el efecto de cada una en cada paso.

Prueba t para los coeficientes

Forward Selection

- Idea principal:
  - Comenzar con un modelo vacío ($Y \sim 1$, solo intercepto).
  - A cada paso, **probar** agregar cada variable fuera del modelo y ajustar el modelo con esa variable agregada.
  - **Seleccionar** la variable que tenga el **menor p-value** (en la prueba t) y que sea $< \alpha$.
  - **Repetir** hasta que no quede ninguna variable que cumpla el criterio de significia.

Prueba t para los coeficientes

Backward Selection

- Idea principal:
  - Comenzar con **todas** las variables en el modelo.
  - Identificar la variable con **p-value más alto** (menos favorable en la prueba t).
  - Si ese p-value es $> \alpha$, eliminar esa variable y ajustar de nuevo.
  - Repetir hasta que todas las variables restantes tengan p-value $< \alpha$.

Prueba t para los coeficientes

Stepwise Selection

- Idea principal:
  - Combina Forward y Backward en un solo procedimiento.
  - En cada paso:

1. Se intenta agregar (Forward) la variable fuera del modelo que tenga el p-value más bajo (si es $< \alpha$).
2. Se intenta eliminar (Backward) la variable dentro del modelo que tenga p-value más alto (si excede α).

Prueba t para los coeficientes

Problemas con la prueba t

- Recordemos que la **prueba t** evalúa la significancia estadística d los coeficientes individuales en un modelo ajustado, bajo la hipótesis nula:
  - $H_0 : \beta_i = 0 \quad$ (la variable X_i no tiene efecto significativo
- Pero este enfoque para la selección de variables no está exento de problemas.

Prueba t para los coeficientes

Problemas con la prueba t

Prueba t para los coeficientes

Problemas con la prueba t

2. **Colinealidad entre variables**:

- En presencia de **multicolinealidad**, los p-values pierden interpretabilidad porque los efectos de las variables se confunden entre sí.
- Esto puede llevar a eliminar variables importantes porque parecen no significativas debido a la redundancia.

Prueba t para los coeficientes

Digresión: Multicolinealidad

- La **multicolinealidad** ocurre cuando dos o más variables independientes están altamente correlacionadas entre sí, lo que dificulta distinguir su impacto individual en la variable dependiente.
- Esto puede causar coeficientes inestables y p-values inflados, haciendo que variables importantes parezcan no significativas debido a la redundancia en la información.
- Por ejemplo, si tenemos dos variables independientes:
  - X_1 : cantidad de horas de ejercicio por semana.
  - X_2 : calorías quemadas por semana.
- Ambas están altamente correlacionadas porque más horas de ejercicio suelen implicar más calorías quemadas. En este caso, el modelo podría no distinguir claramente cuál de las dos variables afecta directamente la variable dependiente (por ejemplo, pérdida de peso).

Prueba t para los coeficientes

Problemas con la prueba t

3. **Optimización basada en predictibilidad vs. significancia:**

- Un modelo con p-values “buenos” **no necesariamente tiene un mejor desempeño predictivo**.
- La capacidad predictiva del modelo no está garantizada solo por la significancia estadística de los coeficientes.

Prueba t para los coeficientes

Problemas con la prueba t

4. **Sobreajuste en conjuntos grandes de variables**:

- Seleccionar variables solo por p-values puede llevar a **sobreajustar** el modelo a la muestra actual, especialmente en conjuntos con muchas variables.
- Esto reduce la capacidad del modelo de generalizar a nuevos datos.

Prueba t para los coeficientes

Conclusión

- La prueba t es una herramienta útil para **interpretar y validar** coeficientes en un modelo ajustado.
- Sin embargo, no está diseñada para seleccionar variables, especialmente en problemas donde el objetivo principal es optimizar el desempeño predictivo del modelo.

¿Qué veremos hoy?

- Prueba t para los coeficientes
- **Selección de variables por métrica**
- Criterios de información: AIC y BIC

Selección de variables por métrica

Selección de variables por métrica

- Otro enfoque para seleccionar variables es a través de la **búsqueda iterativa para minimizar o maximizar una métrica de desempeño**, como el R^2, el RMSE, o cualquier otra métrica relevante para el modelo.
- Este método no se basa en significancia estadística de los coeficientes, sino en optimizar la capacidad predictiva del modelo.

Selección de variables por métrica

Selección de variables por métrica

Feature selection: SFS

- Una herramienta popular para este propósito es el **Sequential Feature Selector (SFS)**, que selecciona variables iterativamente mediante dos estrategias principales:
  1. **Selección hacia adelante (forward)**: Comienza con un modelo vacío y añade variables una por una, evaluando cómo mejora la métrica elegida en cada paso.
  2. **Selección hacia atrás (backward)**: Inicia con todas las variables y las elimina una a una, buscando minimizar el impacto en la métrica.
- Esta funcionalidad está implementada en la librería de Python `sklearn`, la cual, es una de las bibliotecas más populares para machine learning.

Selección de variables por métrica

SFS: Ejemplo

Generamos los datos de entrenamiento y prueba:

```
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import SequentialFeatureSelector

# generamos la matriz de atributos
X = data.drop(columns="target")
# generamos el vector respuesta
y = data['target']

# separamos los datos en entrenamiento y validación
X_train, X_test, y_train, y_test = train_test_split(
  X, y, random_state=42, test_size=0.2
)
```

Selección de variables por métrica

SFS: Ejemplo forward

```python
# creamos el modelo
modelo = LinearRegression()

# configuramos Sequential Feature Selector
sfs = SequentialFeatureSelector(
    modelo,  # modelo base
    n_features_to_select='auto',  # seleccionará automáticamente el nro de
    direction='forward',  # selección hacia adelante
    scoring='neg_mean_squared_error',  # minimizar RMSE
)

# ajustamos el selector a los datos
sfs.fit(X_train, y_train)

# variables seleccionadas
selected_features = X_train.columns[sfs.get_support()]
print("Variables seleccionadas:", list(selected_features))
```

```
Variables seleccionadas: ['sex', 'bmi', 'bp', 's3', 's5']
```

Selección de variables por métrica

SFS: Ejemplo backward

```python
# creamos el modelo
modelo = LinearRegression()

# configuramos Sequential Feature Selector
sfs = SequentialFeatureSelector(
    modelo,  # modelo base
    n_features_to_select='auto',  # seleccionará automáticamente el nro de
    direction='backward',  # selección hacia atrás
    scoring='neg_mean_squared_error',  # minimizar RMSE
)

# ajustamos el selector a los datos
sfs.fit(X_train, y_train)

# variables seleccionadas
selected_features = X_train.columns[sfs.get_support()]
print("Variables seleccionadas:", list(selected_features))
```

```
Variables seleccionadas: ['sex', 'bmi', 'bp', 's1', 's5']
```

¿Qué veremos hoy?

- Prueba t para los coeficientes
- Selección de variables por métrica
- **Criterios de información: AIC y BIC**

Criterios de información: AIC y BIC

Criterios de información: AIC y BIC

- Luego de ver enfoques como la **selección secuencial de variables (SFS)**, otro criterio muy utilizado para la **selección de modelos** son los **criterios de información**, principalmente **AIC** (*Akaike Information Criterion*) y BIC (*Bayesian Information Criterion*).
- Estos criterios se basan en el **balance entre la bondad de ajuste** (log-verosimilitud) y la **complejidad** (número de parámetros) del modelo.

Criterios de información: AIC y BIC

¿Qué es el AIC?

El **AIC (Akaike Information Criterion)** se define como:

$$\mathrm{AIC} = 2k - 2\ln(\hat{L})$$

Criterios de información: AIC y BIC

¿Qué es el BIC?

El **BIC (Bayesian Information Criterion)** se define como:

$$\mathrm{BIC} = \ln(n)\, k - 2\ln(\hat{L})$$

- Donde:
  - n es el número de observaciones (tamaño de la muestra).
  - k y $\ln(\hat{L})$ son los mismos que en el AIC.
- Interpretación principal:
  - A diferencia del AIC, el BIC penaliza más fuertemente el número de parámetros, especialmente cuando n es grande.
  - Al igual que con AIC, **mientras más bajo sea el BIC, mejor**.

Criterios de información: AIC y BIC

Diferencia entre AIC/BIC y el SFS

1. **Fundamento estadístico vs. enfoque predictivo**

- AIC y BIC provienen de la **teoría de la información** (y en el caso del BIC, de la perspectiva bayesiana) y evalúan la **calidad del ajuste** según la log-verosimilitud y la complejidad.
- El **SFS** se centra en optimizar una **métrica de predicción** (por ejemplo, R^2 o RMSE).

Criterios de información: AIC y BIC

Diferencia entre AIC/BIC y el SFS

2. **Selección global vs. selección incremental**

- El AIC/BIC se suele utilizar para **modelos completos** (ej. uno con 2 variables, otro con 3 variables, etc.), y **compararlos** para ver cuál tiene **menor criterio**.
- El SFS hace una búsqueda **iterativa** (añade o quita variables) y se queda con la combinación que **maximiza** la métrica predictiva durante la validación cruzada.

Criterios de información: AIC y BIC

Diferencia entre AIC/BIC y el SFS

3. **Perspectiva estadística vs. predictiva**

- AIC/BIC son muy usados en **estadística clásica** y en **modelos explicativos**, donde se busca la parsimonia sin perder adecuación del modelo.
- El SFS encaja más en entornos de **machine learning** o cuando la meta prioritaria es **predecir**.

Criterios de información: AIC y BIC

Diferencia entre AIC/BIC y el SFS

4. **Utilidad principal de AIC y BIC**

- **Selección de modelos parsimoniosos:** penalizan el número de parámetros, facilitando la elección de un modelo que no sea excesivamente complejo.
- **Comparación de modelos**: al ajustar distintos modelos (con distinto número de variables), se puede **elegir el de menor AIC o BIC**.
- **Control del sobreajuste**: al penalizar la complejidad, reducen la probabilidad de sobreajustar.

Criterios de información: AIC y BIC

Implementación en Python

`statsmodels` ofrece la manera más sencilla de obtener AIC/BIC:

```python
# generamos la matriz de atributos
X = data.drop(columns="target")
# generamos el vector respuesta
y = data['target']

# separamos los datos en entrenamiento y validación
X_train, X_test, y_train, y_test = train_test_split(
  X, y, random_state=42, test_size=0.2
)

# añadimos el intercepto a la matriz de entrenamiento
X_train = sm.add_constant(X_train)
# ajustamos el modelo
modelo = sm.OLS(y_train, X_train)
resultados = modelo.fit()

# reportamos AIC y BIC
print("AIC:", resultados.aic)
print("BIC:", resultados.bic)
```

```
AIC: 3834.201937848667
BIC: 3876.7330864749333
```

Criterios de información: AIC y BIC

Implementación en Python

Comparemos el AIC y BIC del modelo con todas las variables a uno con las variables seleccionadas por SFS backward:

```python
# creamos el modelo
modelo = LinearRegression()

# configuramos Sequential Feature Selector
sfs = SequentialFeatureSelector(
    modelo,  # modelo base
    n_features_to_select='auto',  # seleccionará automáticamente el nro de variables
    direction='backward',  # selección hacia adelante
    scoring='neg_mean_squared_error',  # minimizar RMSE
)

# ajustamos el selector a los datos
sfs.fit(X_train, y_train)

# variables seleccionadas
selected_features = X_train.columns[sfs.get_support()]
print("Variables seleccionadas:", list(selected_features))
```

```
Variables seleccionadas: ['sex', 'bmi', 'bp', 's1', 's2', 's5']
```

Criterios de información: AIC y BIC

Implementación en Python

```python
# generamos la matriz de atributos
X = data.drop(columns="target")
# generamos el vector respuesta
y = data['target']

# dejamos las variables seleccionadas
X = X[selected_features].copy()

# separamos los datos en entrenamiento y validación
X_train, X_test, y_train, y_test = train_test_split(
  X, y, random_state=42, test_size=0.2
)

# añadimos el intercepto a la matriz de entrenamiento
X_train = sm.add_constant(X_train)
# ajustamos el modelo
modelo_2 = sm.OLS(y_train, X_train)
resultados_2 = modelo_2.fit()

# reportamos AIC y BIC
print("AIC:", resultados_2.aic)
print("BIC:", resultados_2.bic)
```

```
AIC: 3829.5239090754194
BIC: 3856.5891854739525
```

Criterios de información: AIC y BIC

Implementación en Python

Comparamos los AIC y BIC de los dos modelos:

```python
print("Modelo 1 (todas las variables):")
print("AIC:", resultados.aic)
print("BIC:", resultados.bic)
print("--"*30)
print("Modelo 2 (con las variables seleccionadas por SFS):")
print("AIC:", resultados_2.aic)
print("BIC:", resultados_2.bic)
```

```
Modelo 1 (todas las variables):
AIC: 3834.201937848667
BIC: 3876.7330864749333
------------------------------------------------------------
Modelo 2 (con las variables seleccionadas por SFS):
AIC: 3829.5239090754194
BIC: 3856.5891854739525
```

Criterios de información: AIC y BIC

Implementación en Python

- En base a los resultados, podemos concluir que el Modelo 2 (con las variables seleccionadas por SFS) es preferible desde el punto de vista de los criterios de información, dado que tiene un AIC y BIC más bajo.
- Tanto el AIC como el BIC penalizan el número de parámetros (complejidad del modelo). Por ello, que el modelo con menos variables (Modelo 2) presente valores menores de ambos criterios indica que logra un mejor equilibrio entre ajuste y parsimonia.
- En otras palabras, conserva el poder explicativo suficiente, pero con menos variables y/o complejidad, lo que reduce el riesgo de sobreajuste y mejora la interpretabilidad del modelo.

Criterios de información: AIC y BIC

- En conclusión, **AIC y BIC** son criterios muy útiles para la **selección de modelos** cuando se prioriza la **parsimonia** y la **buena adecuación estadística**.
- A diferencia del **SFS**, que se centra en optimizar métricas predictivas, AIC/BIC están más ligados a la inferencia estadística y a la teoría de la información, siendo un estándar en muchos enfoques de análisis de datos y estadística clásica.